RSCL — Algorithm 6: Staggered Layered Take-off (2-Colouring of Close Parked Pairs)

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)
Source files described (content hash and state at document build time):

`hu-telepresence-gazebo/festival/control_server_hu.py`	`sha256 3f32578a144c`	2026-11-11 06:32 UTC	clean (HEAD 983eede; last change 10ee643)

1. Purpose and context

After landings the aircraft rest wherever they touched down; two of them may be parked closer than the formation spacing. Launching both simultaneously to the same altitude would place them within the proximity-watchdog distance. Instead of refusing such a take-off, the supervisor computes a layered, staggered launch: aircraft in each close pair are assigned to two altitude layers and to two launch phases. The plan is computed once at command time; release of waiting aircraft happens in the 10 Hz control loop based on telemetry.

2. Inputs, outputs, data structures

- **Inputs:** target names; requested altitude (`params.altitude`, default 1.7 m; the HU route sends 1.65 m); live positions and modes of all five aircraft.
- **Output:** plan `{name: {alt, wait_for: [names]}}` for aircraft in close pairs (empty if none), or a refusal. Each Drone carries `queue_after`, `queue_alt`, `queue_since` and mode `QUEUED` while waiting.
- An event `STAGGERED_TAKEOFF` describes the plan; `STAGGER_RELEASE` / `STAGGER_CANCELLED` record outcomes.

3. Algorithm

Algorithm 6.1 PLAN-STAGGER

Source: festival/control_server_hu.py: Fleet._plan_stagger (called from Fleet.command for action takeoff)

```
function PLAN-STAGGER(targets, params)
  base ← clamp(float(params.altitude or 1.7), 0.25, 2.6 − 0.90)        // ≤ 1.70 m
  L ← [ n ∈ targets : mode(n) ∈ {READY, GROUNDED, WAITING} ]          // aircraft that will launch
  h(a,b) ← horizontal distance
  for n in L, for o in fleet, o ≠ n do
    if mode(o) ∉ PARKED and h(n,o) < 0.88 and z(o) < base + 0.90 + 0.5 then
      refuse "n would climb within h m (horizontal) of airborne o"
  adj(n) ← [ m ∈ L, m ≠ n : h(n,m) < 0.88 − 10⁻⁶ ]
  if all adj empty then return {}                                      // ordinary simultaneous take-off
  colour ← {}
  for root in L (fleet order) with adj(root) ≠ ∅ and root uncoloured do
    colour[root] ← 0; stack ← [root]
    while stack ≠ ∅ do
      a ← pop(stack)
      for b in adj(a) do
        if b uncoloured then colour[b] ← 1 − colour[a]; push(stack, b)
        else if colour[b] = colour[a] then refuse "parked too close together to layer"   // odd cycle
  for n in L with adj(n) ≠ ∅ do
    if colour[n] = 0 then plan[n] ← (alt = base + 0.90, wait_for = [])
    else plan[n] ← (alt = base, wait_for = adj(n))
  return plan
```

Algorithm 6.2 Command application and loop release

Source: festival/control_server_hu.py: Fleet.command (takeoff, hover, land branches), Fleet.loop (QUEUED branch)

```
for n in targets do                                          // inside Fleet.command, action takeoff
  if plan[n].wait_for ≠ ∅ then mode ← QUEUED; queue_after ← wait_for; queue_alt ← plan[n].alt;
queue_since ← now
  else target ← (x, y, plan[n].alt if n ∈ plan else max(z + 0.2, min(2.6, altitude))); mode ← TAKEOFF
every 0.1 s, for each fresh aircraft d with mode QUEUED do     // Fleet.loop
  A ← aircraft in d.queue_after
  if ∀ a ∈ A: fresh(a) and z(a) ≥ 1.20 and not fault(a) then
    target ← (x, y, queue_alt); mode ← TAKEOFF                // STAGGER_RELEASE
  else if now − queue_since > 25 s or ∃ a ∈ A: fault(a) or mode(a) ∈ {LANDING, GROUNDED} then
    mode ← GROUNDED                                           // STAGGER_CANCELLED: stays parked
hover / stop on a QUEUED aircraft → GROUNDED;  land on a QUEUED aircraft → GROUNDED
```

4. Flowchart and state diagram

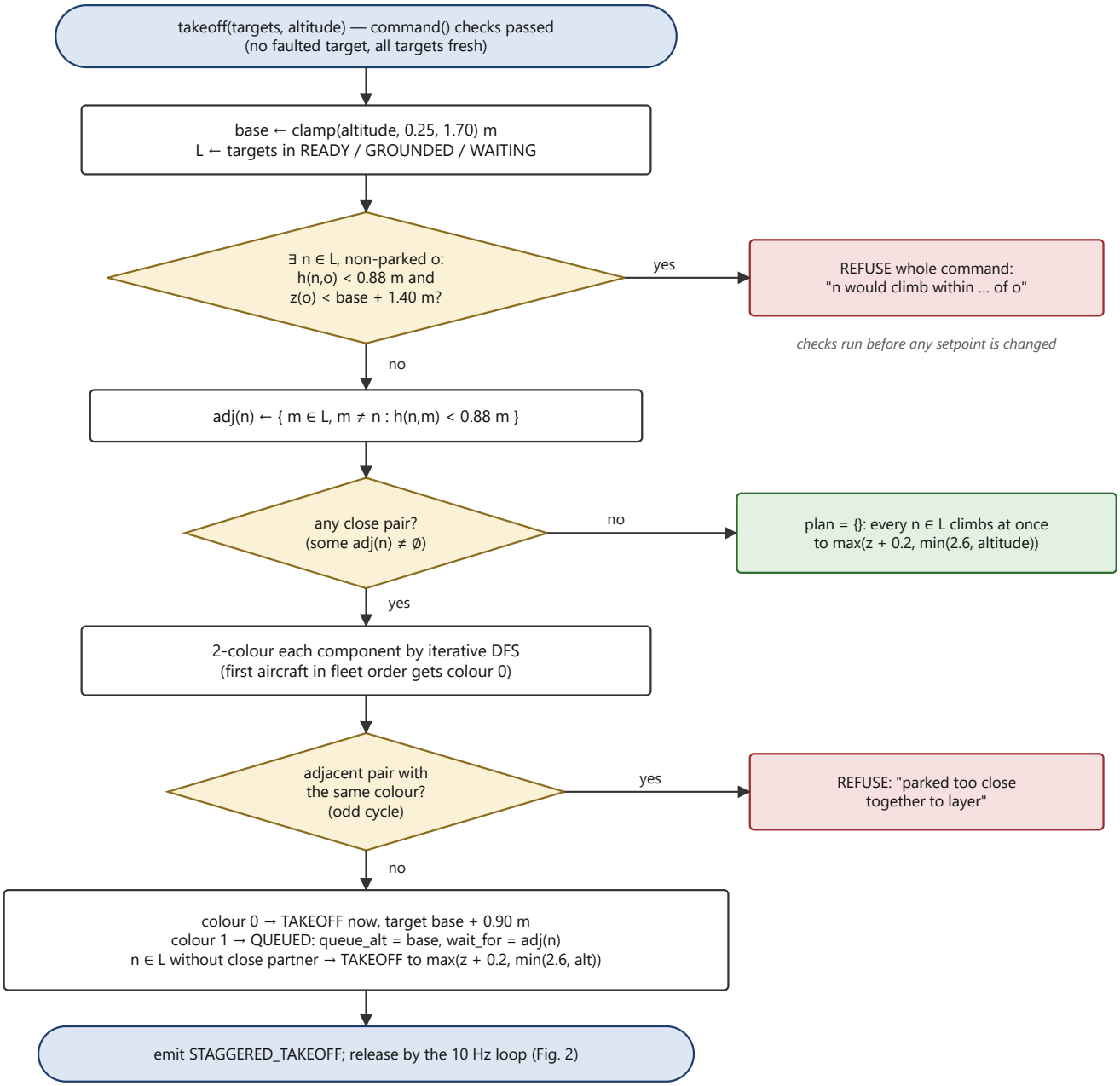


Figure 1. Flowchart of PLAN-STAGGER as reached from `Fleet.command` for action `takeoff`. *h* is horizontal distance; "non-parked" excludes READY, GROUNDED, WAITING and QUEUED. Both refusals raise `ValueError` before any Drone field is written, so the command is rejected as a whole. Thresholds: MIN_SEP = 0.88 m, STAGGER_LAYER_M = 0.90 m.

The take-off has state that persists between control ticks (`mode`, `queue_after`, `queue_alt`, `queue_since`), so its behaviour is also given as a state diagram. It is the take-off subset of the supervisor's per-drone mode FSM, which is drawn in full, with every guard, in the Algorithm 9 document.

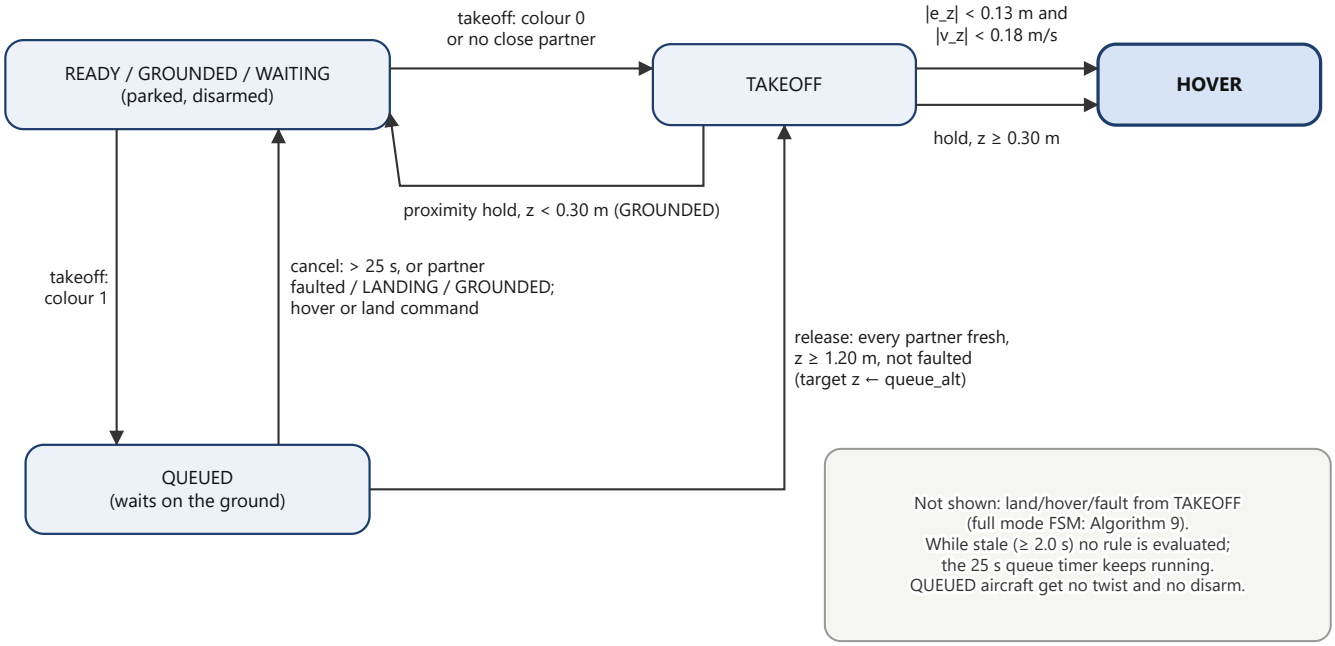


Figure 2. State diagram of the staggered take-off. Guards are those of `Fleet.loop` (QUEUED branch, take-off arrival, proximity hold) and of `Fleet.command` (take-off, hover, land). The release threshold is STAGGER_RELEASE_Z = 1.20 m for every awaited partner.

5. Parameters

Parameter	Value
Conflict distance (horizontal)	MIN_SEP = 0.88 m (strict, with 10^{-6} m tolerance)
Layer offset STAGGER_LAYER_M	0.90 m (code comment: ≥ 1.0 m 3D at 0.5 m horizontal, since $\sqrt{(0.9^2+0.5^2)} \approx 1.03$ m)
Release altitude STAGGER_RELEASE_Z	1.20 m for every awaited neighbour
Queue timeout	25 s
Base altitude	requested altitude clamped to [0.25, 1.70] m; upper layer ≤ 2.60 m
Airborne-overhead guard	any non-parked aircraft within 0.88 m horizontal and below base + 1.40 m refuses the take-off

6. Complexity

Graph construction $O(|L|^2)$; 2-colouring by iterative DFS $O(|L| + |E|)$; airborne check $O(|L|\cdot N)$. With $N = 5$ this is negligible. Release checks are O(deg) per aircraft per 0.1 s tick.

7. Properties

Guaranteed

- Every pair of launching aircraft closer than 0.88 m horizontally ends on different layers 0.90 m apart (a proper 2-colouring), or the whole take-off is refused before any setpoint changes.
- No lower-layer aircraft leaves the ground until all its close neighbours report fresh telemetry at ≥ 1.20 m and are not faulted.
- Odd cycles of close aircraft (e.g. three mutually crowded) are refused with an instruction to launch individually.

- A waiting aircraft never launches after its neighbour aborted, faulted or landed, nor after 25 s; STOP and LAND cancel the wait (no hidden take-off).
- Take-off below an airborne aircraft within 0.88 m horizontally is refused.

Not guaranteed

- Pairs at exactly or just above 0.88 m horizontal launch simultaneously to the same altitude; their spacing then relies on the controller holding XY during the climb.
- The upper layer is base + 0.90 m only for coloured aircraft; uncoupled aircraft use `max(z+0.2, min(2.6, altitude))`, so a requested altitude above 1.70 m produces mixed altitudes.
- The plan uses positions at command time; drift while waiting is not re-planned.
- Colour assignment depends on fleet order (deterministic but arbitrary: the first aircraft of each component climbs high).
- Release depends on the neighbours' altitude only, not on their horizontal drift.
- A QUEUED aircraft receives neither a twist nor an explicit disarm: the QUEUED branch of the loop ends the tick before the 10 Hz disarm that READY/GROUNDED/WAITING aircraft receive. It was disarmed while parked, so it normally stays on the ground, but the “actively disarmed” property of Algorithm 7 does not cover QUEUED.
- While the queued aircraft is stale the release rule is not evaluated for it (the telemetry watchdog runs first), and a stale partner fails the release test; `queue_since` keeps ageing, so a telemetry gap longer than 25 s ends the wait in GROUNDED.
- Rotor downwash is not modelled (see Section 9).

8. Testing and measured results

- Offline: `acceptance/tests/test_v9.py`: `test_close_pair_takeoff_is_layered_not_refused`, `test_three_crowded_aircraft_refused`, `test_stop_cancels_queued_launch`; dry-run behaviour via `test_mission_coordinator.py` (queued aircraft are reported as `staggered_wait_for`).
- Live logs (`logs/hu_festival_events.jsonl`, 2840 lines on 2026-11-12) contain 3 `STAGGERED_TAKEOFF` events, each followed 3–4 s later by its `STAGGER_RELEASE`, and 0 `STAGGER_CANCELLED`: "Horus first to 2.6 m; Ra after Horus to 1.7 m" (released after 3 s), "Horus first to 2.55 m; Ra first to 2.55 m; Thoth after Horus/Ra to 1.65 m" (3 s) and, during the 2026-11-12 formation validation, "Ra first to 2.55 m; Osiris after Ra to 1.65 m" (01:19:11, released 01:19:15). Separation achieved during these launches is **not measured here** as a dedicated result.

9. Comparison with alternatives and related work

Launching several aircraft from crowded parking spots is a small instance of multi-robot motion coordination. Established approaches either serialise the robots by priority and plan each one around the trajectories of higher-priority robots, or compute jointly collision-free, time-parameterised trajectories for the whole team. The HU-MA'AT supervisor does neither: it computes only a launch order and two altitude layers, and releases waiting aircraft on observed altitude.

Approach	Mechanism	What it guarantees (under its assumptions)	Compared with ours
Sequential take-off (operational baseline)	One aircraft at a time; the next starts after a delay or after the previous one reaches altitude.	Simple; works for any number of crowded aircraft.	Ours lets non-conflicting aircraft climb together and needs at most two phases, but refuses odd cycles (three mutually crowded aircraft), which a sequential launch handles.

Prioritized planning [1], [2]	Robots plan one after another in space-time, treating the trajectories of higher-priority robots as moving obstacles.	Collision-free trajectories when every robot finds a plan; incomplete in general, complete for the instance class characterised in [2], which also gives a decentralized variant.	Ours has two priority levels (the colours), only vertical paths and no time-parameterised plan; it makes no completeness claim.
Joint swarm trajectory planning (Crazyswarm line of work) [3], [4], [5]	Discrete multi-agent path finding on a roadmap, then continuous refinement into smooth trajectories; inter-robot constraints use a vertically elongated ellipsoid that models rotor downwash.	Separation of the planned trajectories under that robot model; demonstrated on large teams of nano-quadcopters.	Ours computes no trajectory and does not model downwash: the lower layer climbs to base altitude while its partner hovers 0.90 m above within 0.88 m horizontally. Separation during the climb depends on the controller and on the runtime watchdog.
HU-MA'AT staggered take-off (this document)	2-colouring of the "parked closer than 0.88 m" graph; colour 0 climbs to base + 0.90 m, colour 1 waits until partners are above 1.20 m.	The properties of Section 7 only (layer assignment or refusal; no launch before partners clear 1.20 m).	—

Where ours is weaker. It handles only vertical climbs from rest and only bipartite crowding; it provides no trajectory-level separation guarantee, no downwash model and no timing model; and the colour (which aircraft climbs high) is decided by fleet order rather than by any cost. **Where it is adequate.** With five aircraft in simulation, the decision is computed in microseconds, is explained by a single event line, and the release is gated on telemetry rather than on an assumed climb time. It does not replace a trajectory planner if the fleet or the parking density grows.

10. Relation to other algorithms

Part of the supervisor command path (Algorithm 9 executes the climbs). The proximity watchdog (Algorithm 7) ignores parked and QUEUED neighbours of an aircraft in TAKEOFF. Algorithm 10 models the two launch phases when predicting separations. Algorithm 15 announces QUEUED / release transitions in chat.

11. References

1. M. Erdmann and T. Lozano-Pérez, "On multiple moving objects," *Algorithmica*, vol. 2, pp. 477–521, 1987, doi:10.1007/BF01840371. `https://doi.org/10.1007/BF01840371`
2. M. Čáp, P. Novák, A. Kleiner and M. Selecký, "Prioritized planning algorithms for trajectory coordination of multiple mobile robots," *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 3, pp. 835–849, 2015, doi:10.1109/TASE.2015.2445780. `https://arxiv.org/abs/1409.2399`
3. J. A. Preiss, W. Hönig, G. S. Sukhatme and N. Ayanian, "Crazyswarm: A large nano-quadcopter swarm," *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3299–3304, 2017, doi:10.1109/ICRA.2017.7989376. `https://act.usc.edu/publications/Preiss_ICRA2017.pdf`
4. W. Hönig, J. A. Preiss, T. K. S. Kumar, G. S. Sukhatme and N. Ayanian, "Trajectory planning for quadrotor swarms," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 856–869, 2018. `https://act.usc.edu/publications/Hoenig_TRO2018.pdf`
5. J. A. Preiss, W. Hönig, N. Ayanian and G. S. Sukhatme, "Downwash-aware trajectory planning for large quadrotor teams," *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, doi:10.1109/IROS.2017.8202165. `https://arxiv.org/abs/1704.04852`

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.